

Lab Activity

Week 5: String Immutability and Memory Diagrams

Duration: 70 minutes — Work in pairs!

Name: _____ Partner: _____

Lab Goals

By the end of this lab, you will be able to:

- Predict string behavior using the concept of immutability
- Draw accurate memory diagrams showing string operations
- Distinguish between creating new strings and attempting to modify existing ones
- Debug common string-related errors with confidence

Getting Started

1. Open Thonny on your computer
2. Create a new file called `mod2_week5_lab.py`
3. Save it in your Module2 folder
4. Type your name and your partner's name as comments at the top

1 Exercise 1: String Methods Don't Change Strings! (5 minutes)

Warm-up: Run this code and observe what happens:

```
1 # Test 1: String methods
2 name = "hassan"
3 print("Original name:", name)
4
5 # Try to capitalize
6 name.upper()
7 print("After .upper():", name)
8
9 # Now save the result
10 name = name.upper()
11 print("After saving result:", name)
```

Questions:

1. Why didn't the first `.upper()` change the name? _____
2. What does `.upper()` actually return? _____
3. Complete this rule: String methods return _____, they don't _____ the original.

 **Tip:** Every string method returns a NEW string - you must save it if you want to use it!

2 Exercise 2: Memory Diagram - String Assignment (12 minutes)

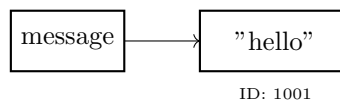
 **Pair Programming:** Switch who types every 5 minutes!

Predict what happens in memory for this code:

```
1 message = "hello"
2 greeting = message
3 message = message + " world"
4 print("message:", message)
5 print("greeting:", greeting)
```

Step 1: Draw the memory diagram for each line:

After line 1: `message = "hello"`



After line 2: `greeting = message`

Draw your diagram here (hint: both variables point to same string!)

After line 3: `message = message + " world"`

Draw your diagram here (hint: a NEW string is created!)

Step 2: Run the code. Did your prediction match? _____

 **Memory Diagram:** When we "change" a string, we actually create a NEW string object!

 **Checkpoint:** Show your memory diagrams to your teacher before continuing.

3 Exercise 3: Trying to Modify Strings (10 minutes)

Experiment: Try to change individual characters in a string:

```
1 # Create a string with a typo
2 word = "pyhton" # Should be "python"
3 print("Original:", word)
4
5 # Attempt 1: Try to fix the typo directly
6 try:
```

```

7     word[2] = 't'    # Try to change 'h' to 't'
8     word[3] = 'h'    # Try to change 't' to 'h'
9 except TypeError as e:
10     print("Error:", e)
11
12 # Attempt 2: Create a new string instead
13 # Fix the typo by building a new string
14 fixed_word = word[:2] + 'th' + word[4:]
15 print("Fixed:", fixed_word)
16 print("Original still:", word)

```

Tasks:

1. Why did Attempt 1 fail? _____
2. In Attempt 2, how many string objects exist after creating `fixed_word`? _____
3. Write code to change "Pakistan" to "Pakestan" (swap 'i' with 'e'):

```

1 country = "Pakistan"
2 # Your code here to create "Pakestan":
3
4
5
6 print(country)    # Should print "Pakestan"

```

4 Exercise 4: String Operations Memory Impact (12 minutes)

Let's trace how many strings are created:

```

1 # Count the string objects created
2 text = "Hi"          # String 1: "Hi"
3 text = text + " "     # String 2: "Hi "
4 text = text + "Ali"   # String 3: "Hi Ali"
5 text = text.upper()   # String 4: "HI ALI"
6 text = text[0:2]      # String 5: "HI"
7
8 # Total: 5 different string objects were created!

```

Now trace this code and count string objects:

```

1 # Your turn - count the strings:
2 name = "sara"         # String 1: _____
3 temp = name.title()   # String 2: _____
4 name = temp           # String count: ____
5 friend = name         # String count: ____
6 name = name + "h"     # String ___: _____
7 friend = friend[0:3]  # String ___: _____

```

Fill in: Total unique string objects created: _____

 **Tip:** Variables pointing to the same string don't create new objects - only operations do!

5 Exercise 5: Memory Diagram - Complex Scenario (12 minutes)

 **Pair Programming:** Switch who types every 5 minutes!

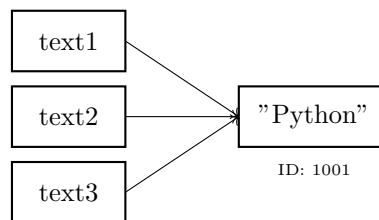
Draw complete memory diagrams for this code:

```

1 # Multiple variables and operations
2 text1 = "Python"
3 text2 = text1
4 text3 = "Python" # Python optimizes - same object!
5
6 text2 = text2.lower() # Creates new string
7 text1 = text1[0:3] # Creates another new string

```

Initial state (after line 3):



Final state (after all operations):

Draw your final memory diagram here:

Questions:

1. How many total string objects exist at the end? _____
2. Which variable(s) still point to "Python"? _____
3. What happens to the original "Python" object? _____

6 Exercise 6: Building Strings Efficiently (10 minutes)

Compare these two approaches for building a string:

Approach A - Inefficient (many intermediate strings):

```

1 # Build a formatted message - creates many strings!
2 result = ""
3 result = result + "Name: " # New string 1
4 result = result + "Ahmed" # New string 2
5 result = result + ", Age: " # New string 3
6 result = result + "13" # New string 4
7 result = result + ", Grade: " # New string 5
8 result = result + "7" # New string 6
9 print(result)

```

Approach B - Efficient (fewer strings):

```

1 # Collect parts, then join once
2 parts = ["Name: ", "Ahmed", ", Age: ", "13", ", Grade: ", "7"]
3 result = "".join(parts) # Only ONE final string created!
4 print(result)

```

Your Task: Convert this inefficient code to use the join pattern:

```

1 # Inefficient version
2 sentence = ""
3 words = ["The", "quick", "brown", "fox"]
4 for word in words:
5     sentence = sentence + word + " " # Many new strings!
6
7 # Your efficient version using join:
8 words = ["The", "quick", "brown", "fox"]
9 # Your code here:
10
11
12 print(sentence)

```

7 Exercise 7: Debug String Confusion (8 minutes)

Find and fix the 3 problems in this code:

```

1 # Program to clean and format user input
2 def clean_input(text):
3     # Remove spaces
4     text.strip() # Problem 1: Result not saved
5
6     # Capitalize first letter
7     text[0] = text[0].upper() # Problem 2: Can't modify string
8
9     # Add period if missing
10    if text[-1] != '.':
11        text + '.' # Problem 3: Result not saved
12
13    return text
14
15 # Test the function
16 user_text = " hello world "
17 cleaned = clean_input(user_text)
18 print(f"Cleaned: '{cleaned}'")

```

Fixed version:

```

1 def clean_input(text):
2     # Fix 1: Save the result of strip()
3     text = -----
4
5     # Fix 2: Create new string with capital first letter
6     text = -----
7
8     # Fix 3: Save the concatenation result
9     if text[-1] != '.':
10        text = -----
11
12    return text

```

✔ **Checkpoint:** Test your fixed function with different inputs!

8 Exercise 8: String vs List Comparison (8 minutes)

Run this comparison to see the difference between immutable strings and mutable lists:

```

1  # Part A: Strings (immutable)
2  str1 = "hello"
3  str2 = str1
4  str1 = str1.upper()
5  print("String Results:")
6  print(f"    str1: {str1}")    # "HELLO"
7  print(f"    str2: {str2}")    # Still "hello"!
8
9  # Part B: Lists (mutable - preview of next week!)
10 list1 = ['h', 'e', 'l', 'l', 'o']
11 list2 = list1
12 list1[0] = 'H'    # This DOES work with lists!
13 print("\nList Results:")
14 print(f"    list1: {list1}")    # ['H', 'e', 'l', 'l', 'o']
15 print(f"    list2: {list2}")    # Also changed!

```

Reflection Questions:

1. Why didn't `str2` change when we modified `str1`? _____
2. Why DID `list2` change when we modified `list1`? _____
3. Which is safer from unexpected changes: strings or lists? _____

9 Lab Summary

✔ **Checkpoint:** Final checkpoint - make sure you have:

- ☐ Completed all exercises with your partner
- ☐ Drew memory diagrams for Exercises 2 and 5
- ☐ Fixed the buggy code in Exercise 7
- ☐ Understood why strings can't be modified (immutability)
- ☐ Saved your file as `mod2.week5.lab.py`

Reflection (3 minutes)

Rate your understanding of today's concepts:

Concept	🙄 Need Help	😬 Getting It	😄 Got It!
String immutability	☐	☐	☐
Memory diagrams	☐	☐	☐
Creating new strings	☐	☐	☐
String method behavior	☐	☐	☐

The biggest "aha!" moment about strings today:

One thing that still confuses me: _____

True or False (circle one):

1. Strings can be modified after creation: **True** / **False**
2. `text.upper()` changes the original text: **True** / **False**
3. Every string operation creates a new object: **True** / **False**
4. Two variables can point to the same string: **True** / **False**

🌱 Level Up! You now understand the secret of strings!